

PRÁCTICA

1) (3 Puntos) A partir del esquema dado, escriba las sentencias SQL necesarias para resolver los ítems propuestos:

Carreras(cod_carrera: integer, nombre: string, duracion: integer)

Alumnos(numa: integer, nombrea: string, carrera: integer, nivel: string, edad: integer, cursados: integer)

CF: carrera → Carreras(cod_carrera)

Cursos(nombre: string, impartido en: string, aula: string, idp: integer, carrera: integer)

CF: idp → Profesor(idp); carrera → Carreras(cod_carrera)

Matriculado(numa: integer, nombrec: string, aprobado: booleano)

CF: numa → Alumnos(numa); nombrec → Cursos(nombre)

Profesores(idp: integer, nombrep: string, iddep: integer)

- a) Crear un trigger tal que, al cargar una nueva matriculación, controle que el alumno esté inscripto en la misma carrera a la que pertenece el curso, cancelando la acción y emitiendo un error en caso contrario.

delimiter //

```
CREATE TRIGGER control_carrera
BEFORE INSERT ON Matriculado /* PODRÍA CONTEMPLARSE TAMBIÉN AGREGAR OTRO CON
"BEFORE UPDATE" */
FOR EACH ROW
BEGIN
    SET @carrera_alumno = 0;
    SET @carrera_curso = 0;
    SELECT a.carrera INTO @carrera_alumno
        FROM Alumnos a
        WHERE a.numa = NEW.numa;
    SELECT c.carrera INTO @carrera_curso
        FROM Cursos c
        WHERE c.nombre = NEW.nombrec;
    IF @carrera_alumno <> @carrera_curso THEN
    BEGIN
        SET @mensaje = CONCAT('La carrera del alumno (', @carrera_alumno,
        ') es distinta a la del curso (', @carrera_curso, ')');
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = @mensaje;
    END;
    END IF;
END//
DELIMITER ;
```

- b) Asumiendo que el campo “*cursados*” de la tabla “**Alumnos**” contabiliza los cursos que cada alumno tiene aprobados (en la tabla “**Matriculado**” el campo “*aprobado*” se actualiza a “TRUE” cuando el alumno aprueba el curso correspondiente): Crear un trigger para actualizar el contador cada vez que un alumno aprueba un curso;

delimiter //

```
CREATE TRIGGER actualizar_cant
AFTER UPDATE ON Matriculado
FOR EACH ROW
BEGIN
    IF NEW.aprobado THEN
        UPDATE Alumnos
```

```
        SET cursados = cursados + 1
        WHERE numa = NEW.numa;
    END IF;
END//
```

```
delimiter ;
```

- c) Crear una rutina almacenada que, dado el *idp* de un profesor, liste los detalles de los cursos que dicta y los alumnos que están matriculados y aún no han aprobado, ordenados por nombre de los cursos y nombre de los alumnos.

```
delimiter //
```

```
CREATE PROCEDURE ListarCursos(IN profesor SMALLINT)
BEGIN
    SELECT c.nombre, c.impartido_en, c.aula, a.nombrea, a.nivel, a.edad
    FROM Cursos c, Alumnos a, Matriculado m
    WHERE c.nombre = m.nombrec AND a.numa = m.numa AND c.idp = profesor AND NOT
    m.aprobado
    ORDER BY m.nombrec, a.nombrea;
END//

delimiter ;
```

2) (3 puntos) Seguridad.

Usando el esquema de la BD “INSTITUTO” dado en el ejercicio anterior, escriba las sentencias SQL que permitan realizar las siguientes acciones:

- a) Crear un nuevo usuario llamado Juan y asignar la contraseña “Juan123”. La cuenta debe estar bloqueada inicialmente.

```
CREATE USER 'Juan'@'%' IDENTIFIED BY 'Juan123' ACCOUNT LOCK;
```

- b) Asignar permiso para realizar únicamente consultas de visualización de las columnas “nombrea”, “nivel” y “cursados” de la tabla “Alumnos” en la base de datos “INSTITUTO”.

```
GRANT SELECT (nombrea, nivel, cursados) ON INSTITUTO.Alumnos TO 'Juan'@'%' ;
```

- c) Crear un usuario que tenga los privilegios necesarios para respaldar y restaurar la base de datos “INSTITUTO”.

```
CREATE USER 'respaldo'@'%' IDENTIFIED BY 'Respaldo123';
GRANT SELECT, CREATE, ALTER, DROP, INSERT ON INSTITUTO.* TO 'respaldo'@'%' ;
/* ESTOS PRIVILEGIOS AL MENOS PARA RESGUARDAR DATOS Y ESTRUCTURAS DEL EJERCICIO
Y PODER VOLVER A CREAR Y CARGAR */
```

3) (2 puntos) Respaldo y restauración.

- a) Plantee un esquema de respaldos para la base de datos “INSTITUTO” del ejercicio 1.

DEBERÍA HABER UN PLANTEO SOBRE POSIBLES VOLÚMENES DE DATOS/TRANSACCIONES, Y QUE LA CANTIDAD Y TIPO DE RESPALDOS DIARIOS/SEMANALES SEAN COHERENTES CON DICHS VOLÚMENES.

- b) Utilizando el usuario creado en 2.c, escriba las sentencias/comandos, y aclaraciones que considere necesarias, tanto para hacer respaldos (completos y/o incrementales) como para recuperar los datos en caso de una contingencia.

PARA REALIZAR UN RESPALDO LÓGICO COMPLETO QUE REINICIE LOS BINARY LOGS PARA LLEVAR RESPALDOS INCREMENTALES:

```
mysql-dump -u respaldo -p --single-transaction --flush-logs  
--source-data=2 INSTITUTO > instituto.sql
```

PARA RECUPERAR EL RESPALDO COMPLETO:

```
mysql -u respaldo -p < instituto.sql
```

PARA RECUPERAR TRANSACCIONES DE LOS BINARY LOGS:

```
mysqlbinlog binlog.000009 binlog.000010 | mysql -u respaldo -p  
SE RECUPERA LOS BINLOGS NECESARIOS HASTA EL PUNTO DE FALLO, Y DE SER NECESARIO  
SE AGREGAN/DESCARTAN FRAGMENTOS ESPECÍFICOS MEDIANTE LAS OPCIONES --start-posi-  
tion / --stop-position
```

TEORÍA (2 puntos)

1. Seguridad en bases de datos. Definición de Políticas de Contraseñas. Tipos de privilegios. Roles y Grupos.
2. Replicación en bases de datos relacionales. Describa los tipos de réplica que conoce. Explique en qué casos es útil la réplica. ¿Reemplaza al esquema de backup? ¿Por qué?
3. Funcionalidad de soporte en RDBMS. Algoritmo ARIES. Explique las 3 fases de recuperación.
4. Control de concurrencia. ¿Qué es una planificación en serie? ¿Qué es una planificación serializable? ¿Por qué se considera que una planificación en serie es correcta? ¿Por qué se considera que una planificación serializable es correcta?